Using Git to Manage Projects and Build a Project on GitHub

What is Git?

- Git is a distributed version control system.
- Tracks changes in source code during software development.
- Allows multiple developers to collaborate on a project.
- Key features: Branching, Merging, and History.

What is GitHub?

- GitHub is a web-based platform for hosting Git repositories.
- Provides a collaborative environment for developers.
- Key features: Pull Requests, Issues, GitHub Pages, and Actions.

Setting Up Git and GitHub

- Installing Git on your system.
- Configuring Git with your username and email.
- Creating a GitHub account.
- Generating SSH keys for secure communication.

Creating a New Repository

- Initialise a new repository locally with **git init**. The command can also be used to convert an existing, unversioned project to a Git repository or initialise a new, empty repository.
- Create a new repository on GitHub.
- Connect the local repository to remote GitHub with **git remote add origin**.
- Push the initial commit to GitHub.

Basic Git Commands

- **git status**: Check the status of your repository.
- **git add**: Stage changes for commit (adds a change in the working directory to the staging area).
- **git commit**: Save changes to the repository.
- **git push**: Push changes to a remote repository.
- **git pull**: Fetch and merge changes from a remote repository.

- **What is a branch?** A pointer to a snapshot of your changes. Branches allow you to develop features, fix bugs, or safely experiment with new ideas in a contained area of your repository. Always create a branch from an existing branch. Create a new branch from the default branch of your repository.

- Creating a new branch:
  - **git branch <branch-name>**.
- Switching branches (navigate between the branches):
  - **git checkout <branch-name>.**
- Merging branches (add changes from one branch to another branch):
  - **git merge <branch-name>.**
- Deleting a branch: **git branch -d <branch-name>.**

Collaborating with GitHub

- Forking a repository.
  - A **fork** is a new repository that shares code and visibility settings with the original repository.
  - **Forking** is a making copy of the main repository under your GitHub account to make modifications.
  - Any changes made to the **original repository** will be reflected back to your forked repositories.
  - Any changes to your **forked repository** you will have to **explicitly create** a **pull request** to the original repository.
    - If your pull request is approved by the administrator of the original repository, then your changes in the forked repository will be committed/merged with the existing **original** code base.
    - **Anyone can** fork an existing repository and ***push*** changes to their personal fork without requiring access to be granted to the source repository. The changes must then be ***pulled*** into the source repository **by the project maintainer or administrator**.

- Cloning a repository: **git clone <repository-url>.**
- Creating and reviewing Pull Requests.
- To resolve merge conflicts in Git, you must open the file and make the changes (manually editing the conflicting files).

Building a Project on GitHub

- Creating a project structure.
- Adding and committing files.
  - **Committing files**: add changes to the history of the repository and assign a commit name to it.
- Pushing the project to GitHub.
- Using **GitHub Pages** for project hosting: GitHub Pages is a static site hosting service that takes HTML, CSS, and JavaScript files straight from a repository on GitHub, optionally runs the files through a build process, and publishes a website
- **GitHub Actions** allows users to automate tasks within a GitHub repository by creating workflows. **Workflows** are a series of tasks that are automatically performed when the workflow runs.

Best Practices

- Writing meaningful commit messages.
- Keeping commits small and focused.
- Regularly pulling changes from the remote repository.
- Reviewing code before merging.

Building a Project on GitHub

- Creating a project structure.
- Adding and committing files.
- Pushing the project to GitHub.
- Using GitHub Pages for project hosting.
- Automating workflows with GitHub Actions.

Best Practices

- Writing meaningful commit messages.
- Keeping commits small and focused.
- Regularly pulling changes from the remote repository.
- Reviewing code before merging.